

A Memoryless Reactive Control System for Wall-Following and Autonomous Centering

Joana Pimenta, Mihai Mărtinaş, João Proença

Faculdade de Engenharia da Universidade do Porto (FEUP)

Porto, Portugal

up202206120@up.pt, up202512406@up.pt, up202207835@up.pt

Abstract—This paper presents a purely reactive, memoryless ROS 2 mobile robot designed to autonomously navigate a “5-shaped” maze, identify an inner circular room, and position itself at the geographic center facing the exit. The decoupled, three-node architecture (radar, decision, movement) relies entirely on instantaneous, discretized LiDAR data without global localization or mapping. Building upon a Spatial Proportional-Derivative (PD) controller for robust left-hand wall-following and instantaneous geometric routing logic, several advanced mechanisms were implemented to fulfill Extra Merit objectives. These features, ranging from stateless exploration strategies to hybrid kinematic limiters, are designed to enhance the robot’s hardware realism and scale-invariant shape recognition while strictly adhering to the memoryless constraints of the project. The system was validated across two distinct “5-shaped” maps.

Index Terms—Robot, ROS 2, Control Systems, PID Control, Reactive Navigation, Wall-following

I. INTRODUCTION

Traditional mobile robotics often relies on computationally heavy algorithms for mapping, localization, and navigation. However, many applications benefit from reactive control systems, which offer a highly efficient, lightweight alternative by establishing direct mappings between instantaneous sensor inputs and actuator outputs. This work explores the capabilities and limitations of such memoryless systems.

Specifically, this study focuses on developing a purely reactive, differential-drive robot in ROS 2, simulated within Gazebo. Its mission is to autonomously navigate a “5-shaped” maze, definitively identify an inner circular room, position itself at the geographic center, and orient toward the exit, all without relying on global localization, explicit internal maps, or persistent memory.

The constraints imposed on the development of this project are strictly defined to test the limits of reactive navigation:

- Design and test a reactive differential-drive robot under the ROS 2 framework.
- The robot must operate without any form of memory or global state tracking, with the sole exception being the permissible use of a Proportional-Integral-Derivative (PID) controller.
- Navigation must rely entirely on instantaneous local sensor data. No “cheating” is allowed. This means no GPS, no global localization maps, and no privileged information from the simulator.

- The simulation environment consists of a “5-shaped” wall structure containing an inner circular room.
- The robot must start at a random position and heading outside the structure, follow the wall, detect the entrance to the circular room, and ultimately stop at the room’s exact center facing outward.

Based on these design objectives and constraints, the main contributions of this work can be summarized as follows:

- Decoupled ROS 2 Architecture: Implementation of a three-node system (Radar, Decision, Movement) utilizing a computationally light 36-bin discretized LiDAR array.
- Stateless Exploration: A memoryless wandering reflex utilizing temporal sine waves to guarantee wall intersection from isolated spawn points.
- Spatial PD Control: A robust left-hand wall-following controller that derives dampening errors spatially rather than temporally.
- Strict Circularity Detection: A mathematical filter that reliably distinguishes circular rooms from square ones. By evaluating only the 20th to 80th percentiles of the LiDAR data, the algorithm safely ignores the long laser rays escaping through the open doorway, preventing false-positive polygon detections.
- Hybrid Speed and Acceleration Limiting: A two-part system that respects physical motor limits without violating the “no memory” software rule. Instantaneous Python math prevents the wheels from exceeding their maximum top speed, while hardware-level torque constraints in the SDF naturally create smooth, realistic acceleration ramps through simulated physics.

The remainder of this paper is organized as follows. Section II provides a brief overview of the state of the art in reactive control systems and autonomous wall-following. Section III describes the proposed system architecture, detailing the LiDAR discretization, PI(D) control laws, centering heuristics and additional features. Section IV outlines the experimental setup. Section V presents the simulation results and discusses the performance and limitations of the approach. Finally, Section VI concludes the paper and outlines potential directions for future work.

II. RELATED WORKS

Autonomous robot navigation has historically been divided into deliberative, reactive, and hybrid paradigms [1]. Tradi-

tional deliberative architectures rely on computationally heavy "Sense-Plan-Act" models, which require comprehensive internal maps and struggle to react quickly to dynamic changes. To address these limitations, reactive control systems emerged as a lightweight, memoryless alternative. A foundational contribution is Rodney Brooks' Subsumption Architecture [2], which demonstrated that intelligent behavior could emerge from layered sensor-to-action mappings, allowing higher-level behaviors to override lower-level reflexes. Arkin [3] further expanded this through behavior-based vector summation.

A classic application of reactive control is autonomous wall-following. While basic conditionals suffice for simple obstacle avoidance, smooth navigation typically requires closed-loop feedback via a Proportional-Integral-Derivative (PID) controller. Although PID error tracking introduces a localized form of transient memory, it remains a permissible standard for stable kinematic control in reactive systems. Today, modern reactive architectures leverage middleware like ROS2 to decouple behaviors into modular nodes. By discretizing 360-degree LiDAR point clouds into localized bins, contemporary systems can compute advanced spatial heuristics, such as doorway identification and geometric centering, entirely on the fly, achieving complex spatial navigation while strictly preserving the memoryless constraint.

III. SYSTEM ARCHITECTURE AND IMPLEMENTATION

This section outlines the structural framework of the autonomous system, detailing the simulated testing ground, the robot's physical configuration, and the decoupled ROS 2 node architecture designed to achieve purely reactive navigation.

A. Simulation Environment

The development and testing of the reactive control system were conducted using the Gazebo simulator, fully integrated with the ROS 2 framework. To satisfy the specific constraints of the mission, a custom world was designed featuring a "5-shaped" wall structure that encapsulates an inner circular room. To prove the scale-invariance and robustness of the reactive algorithms, the system was validated across two distinct variations of this "5-shaped" map (Figures 1 and 8). In all tests, the robot is spawned at a random starting position and heading outside of the structure, forcing it to dynamically locate the entrance rather than relying on hardcoded trajectories.

The robot utilized for this experiment, shown in Figure 2, is modeled as a standard differential-drive mobile platform, characterized by two independently driven wheels on a common axis. To fulfill the Extra Merit requirements for limited acceleration and angular velocity (ω), hardware-level constraints were integrated into the robot's SDF model. By capping the motor torque at 0.8 Nm, the robot is physically prevented from accelerating instantly, creating a natural, smooth movement ramp. Additionally, the maximum wheel spin was restricted to 6.0 rad/s, which strictly enforces a top speed of 0.3 m/s and prevents the robot from reaching unstable angular velocities during turns.

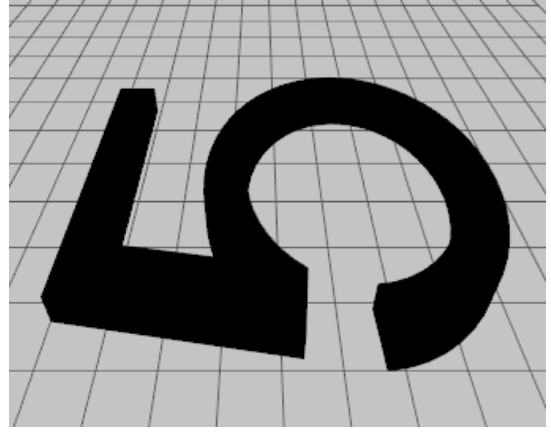


Fig. 1. Overhead view of the "5-shaped" simulation environment in Gazebo, featuring the outer walls and inner circular room.

Because the primary constraint of this project mandates a purely reactive, memoryless approach, the robot is equipped with a single exteroceptive sensor: a 2D LiDAR scanner. While the raw sensor provides 360-degree range data, this is heavily discretized by the software into 36 directional bins to reduce computational overhead.

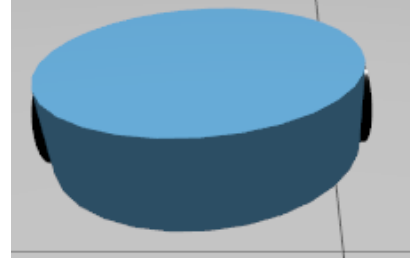


Fig. 2. The differential-drive mobile robot used in the simulation, equipped with a 360-degree LiDAR sensor.

Crucially, to ensure strict adherence to the assignment's constraints, all forms of global localization, GPS, and mapping were disabled. Furthermore, proprioceptive data such as wheel encoders and standard ROS 2 odometry (`/odom`) transforms were intentionally excluded from the control logic. The robot operates completely blind to its global coordinates, relying entirely on the instantaneous physical feedback provided by the LiDAR to navigate the maze and identify the central room.

B. Controllers Design

The autonomous navigation is governed by a memoryless subsumption architecture that transitions between four behavioral states based on real-time LiDAR transformations.

1) *Wall-Following (Spatial PD Control)*: When a wall is detected within a 0.8m, the robot engages a left-hand wall-following behavior. To maintain a target distance (d_{target}) of 0.5m without utilizing temporal memory, a Spatial PD Controller is used.

The PD gains were determined empirically through iterative testing in the Gazebo simulation environment. Starting from an

initial estimate of $K_P = 1.0$ and $K_D = 0.5$, the proportional gain was first increased until the robot consistently maintained the target wall distance without oscillation. The derivative gain was then adjusted to dampen overshoots at corners and sharp geometry changes. The final values of $K_P = 1.5$ and $K_D = 0.8$ were selected as the best trade-off between tracking accuracy and stability.

The proportional error (e) is derived from the minimum left-side distance, while the derivative component (dampening) is calculated spatially by comparing the difference between forward-left and backward-left rays ($d_7 - d_{11}$).

$$e(t) = d_{meas} - d_{target} \quad (1)$$

The angular velocity (ω) is defined as:

$$\omega = (1.5 \times e) + (0.8 \times \text{spatial_heading_error}) \quad (2)$$

Linear velocity is adaptively scaled, reducing speed during sharp corrections to maintain kinematic stability.

2) *Room Identification and Strict Circularity*: To differentiate circular rooms from false-positive geometries like square bays, the robot performs an instantaneous Least Squares Regression to fit a circle to the LiDAR point cloud. This provides a candidate center (x_c, y_c) and radius R . To ensure the stop condition is scale-invariant and ignores the open doorway, a Percentile Variance algorithm is applied. By evaluating the normalized variance between the 20th and 80th percentiles:

$$V = \frac{p_{80} - p_{20}}{R} \quad (3)$$

The robot effectively "trims" outliers caused by doorframes and sensor noise. A threshold of $V < 0.15$ is used to mathematically confirm the circularity of the room before triggering the centering and exit-alignment behaviors.

3) *Precision Centering and Exit Alignment*: Once inside the circle, the robot utilizes the calculated geometric center (c_x, c_y) from the regression analysis.

- **Centering**: The robot steers toward the geometric center until the distance to center is less than 0.15m (margin of error).
- **Exit Alignment**: The robot identifies the exit by calculating the mean angular vector of the maximum LiDAR returns (the doorway). It executes a zero-radius turn until its heading error relative to the exit is less than 0.2 rad.
- **Stop Condition**: When both the 0.15m centering tolerance and the 0.2 rad alignment tolerance are met simultaneously, a high-priority stop flag is published, terminating the mission.

4) *Stateless Wandering (Extra Merit)*: If no obstacles are detected within 7.9m (e.g., when spawned in an open void), the robot enters a wandering state. To satisfy the extra merit requirement for initial randomness without utilizing stateful search patterns, it injects a temporal sine wave into the angular velocity command:

$$\omega(t) = 0.8 \sin(t) + 0.2 \quad (4)$$

This generates a sweeping search pattern of overlapping, asymmetrical loops that mathematically guarantees eventual wall intersection using zero memory.

C. Implementation Details

To adhere to best practices in robotic software engineering, the system's logic is decoupled into a three-node ROS 2 architecture. This modular design allows sensor processing, decision-making, and kinematic execution to occur asynchronously through a unidirectional pipeline, as illustrated in the computation graph in Figure 3.

1) *Radar Node (Sensor Processing)*: The RadarNode acts as the system's perception filter. This node subscribes to high-frequency `/scan` messages. To reduce the computational load on the behavioral logic, it discretizes 360-degree LiDAR data into 36 uniform 10-degree bins, selecting the minimum distance for each and filtering out invalid readings. The resulting low-dimensional array is published to `/processed_scan`.

2) *Decision Node (Behavioral Logic)*: Serving as the system's "brain," this node evaluates the 36-bin array against hierarchical priorities: centering, wall-following, or wandering. Strictly memoryless, every callback execution represents an isolated state calculation. The computed velocities are packaged into a custom `geometry_msgs/Vector3` message (where x represents angular velocity (ω), y is linear velocity (v), and z serves as a stop flag) and published to `/robot_command`.

3) *Movement Node (Kinematic Execution)*: This node translates behavioral intents into actual motion. It subscribes to `/robot_command` and maps the custom vector into a standard `geometry_msgs/Twist` message for the Gazebo differential drive plugin. If the stop flag ($z = 1.0$) is received, the node forces all velocities to zero to terminate the mission.



Fig. 3. The ROS 2 computation graph illustrating the unidirectional data flow from the LiDAR sensor through the custom processing nodes to the differential drive controller.

IV. EXPERIMENTS

The system was validated in the Gazebo simulation environment using the "5-shaped" map described in Figure 1. The robot was spawned at five distinct starting positions and headings outside the structure, with no prior knowledge of its location or orientation relative to the maze. All runs used identical software parameters.

For each trial, the mission time was measured from the first log entry to the `Mission Complete` stop signal. The starting positions are described qualitatively, as no global localization is available to the robot.

To evaluate the consistency of the system, five additional runs were performed from the same fixed starting position ($x = -7.4\text{m}$, $y = 1.8\text{m}$, $\theta = 0.0\text{rad}$), shown in Figure 4. Table II summarizes the mission completion times alongside

the control and kinematic parameters used throughout all experiments.

V. RESULTS AND DISCUSSION

The system successfully completed the mission in all tested configurations. The trajectory shown in Figure 5 confirms the expected behavior: a structured wall-following path along the left side of the structure, and a smooth convergence toward the center of the circular room. The system was additionally tested on a second, differently shaped map, confirming the scale-invariance of the circularity detection algorithm.

Figure 5 illustrates the trajectory followed by the robot during one of the repeatability test runs from the fixed starting position ($x = -7.4$ m, $y = 1.8$ m, $\theta = 0.0$ rad), recorded using ground truth position data from the Gazebo simulator.

Figure 6 shows the evolution of the distance error and heading error during the wall-following phase. The distance error represents the difference between the measured left-side wall distance and the target of 0.5 m, while the heading error captures the angular misalignment of the robot relative to the wall, computed spatially as the difference between the forward-left and backward-left LiDAR rays. Ideally, both errors should converge to zero during steady-state wall-following, with transient spikes expected at corners and geometry changes.

Figure 7 illustrates the centering phase once the robot has entered the circular room. The upper plot shows the distance from the robot to the geometric center estimated by the least squares regression, with the 0.15 m acceptance threshold marked in red. The lower plot shows the heading error relative to the exit doorway, with the 0.2 rad acceptance threshold. The mission is considered complete only when both conditions are simultaneously satisfied.

All five runs across different starting positions completed successfully, with mission times ranging from 176 s to 296 s. The longest run corresponds to the starting position on the left side of the structure, which required the robot to travel further before locating a wall. The repeatability tests demonstrate consistent performance with a mean mission time of 53.4 s and a standard deviation of only 2.7 s, confirming the reliability of the system under identical initial conditions.

Several limitations of the proposed approach were identified during testing. The most significant is the exclusive left-hand wall-following strategy, which, while guaranteeing eventual maze traversal, does not always produce the shortest path to the circular room. Depending on the robot's initial orientation relative to the structure, the left-hand rule may cause the robot to navigate the longer perimeter of the "5-shaped" maze before reaching the entrance, as observed in Run 4 (296 s) compared to Run 2 (176 s). A right-hand rule or a hybrid strategy that selects the shorter path based on instantaneous sensor data could mitigate this behavior, but would require additional logic that may compromise the simplicity of the reactive architecture.

A second limitation is the dependence of the wandering phase on the initial distance to the structure. Since the

robot has no global awareness, it relies entirely on the sinusoidal sweep pattern to locate a wall, which introduces non-deterministic behavior in open environments. Runs starting further from the structure naturally result in longer mission times, as observed in the variable-position tests.

Finally, the purely reactive nature of the system makes it susceptible to edge cases at sharp corners and narrow passages, where the PD controller may produce transient oscillations before stabilizing. While these oscillations are self-correcting in most scenarios, they could potentially cause the robot to lose wall contact in environments with more complex geometries.

VI. CONCLUSION

This work presented the design, implementation, and validation of a purely reactive, memoryless mobile robot capable of autonomously navigating a "5-shaped" maze, identifying an inner circular room, and positioning itself at the geometric center facing the exit. The system was implemented under the ROS 2 framework and simulated in Gazebo, using a decoupled three-node architecture consisting of a radar node for LiDAR discretization, a decision node for behavioral logic, and a movement node for kinematic execution.

The behavioral logic is governed by a subsumption architecture with four prioritized layers: a stateless sinusoidal wandering reflex for open spaces that transitions to a targeted wall approach upon obstacle detection, a left-hand Spatial PD wall-following controller, a scale-invariant least squares circle detection algorithm with p80-p20 percentile variance filtering, and a geometric centering and exit alignment behavior. Crucially, all behaviors operate exclusively on instantaneous LiDAR data, with no persistent memory, no global localization, and no privileged simulator information.

The system successfully completed the mission in all tested configurations across five distinct starting positions and two different map scales, demonstrating robustness to initial conditions and scale variance. Repeatability tests from a fixed starting position confirmed highly consistent performance with a mean mission time of 53.4 s and a standard deviation of 2.7 s. The circularity detection algorithm proved reliable across all runs, with no false positives during wall-following and no missed detections upon entering the circular room.

The main limitation identified is the exclusive left-hand wall-following strategy, which does not always produce the shortest path to the circular room entrance, resulting in mission time variability across different starting positions. Additionally, the purely reactive nature of the system makes it susceptible to oscillations at sharp corners and dependent on the initial distance to the structure during the wandering phase.

Future work will focus on implementing a hybrid wall-following strategy that dynamically selects the shorter path based on instantaneous sensor data, as well as exploring more sophisticated wandering behaviors to reduce the time spent in open environments. The integration of additional sensors and the extension to multi-robot scenarios are also promising directions for further development.

APPENDIX

TABLE I
MISSION RESULTS ACROSS DIFFERENT STARTING POSITIONS

Run	Start Position	Time (s)	Success
1	Right of structure, far	212	Yes
2	Right of structure, close	176	Yes
3	Above structure, far	183	Yes
4	Left of structure, close	296	Yes
5	Above structure, close	186	Yes

TABLE II
MISSION TIMES AND CONTROL PARAMETERS — FIXED STARTING POSITION ($x = -7.4$ M, $y = 1.8$ M, $\theta = 0.0$ RAD)

Mission Times (s)	
Run 1	56
Run 2	49
Run 3	53
Run 4	53
Run 5	56
Mean $\pm$ Std Dev	53.4 ± 2.7
Control Parameters	
K_P	1.5
K_D (spatial)	0.8
Commanded Speed	1.0 m/s
Effective Max Speed	0.3 m/s
Max Angular Velocity	5.0 rad/s
Max Acceleration (torque limit)	0.8 Nm

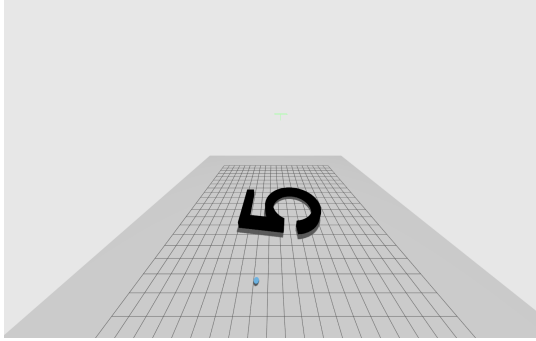


Fig. 4. Fixed starting position for repeatability test ($x = -7.4$ m, $y = 1.8$ m, $\theta = 0.0$ rad).

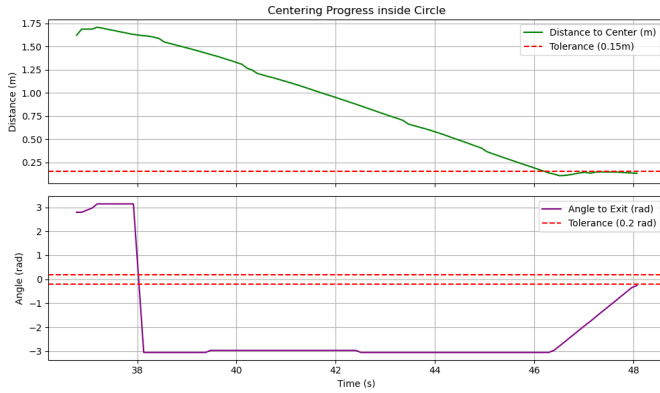


Fig. 7. Centering progress inside the circular room: distance to geometric center and heading error relative to the exit.

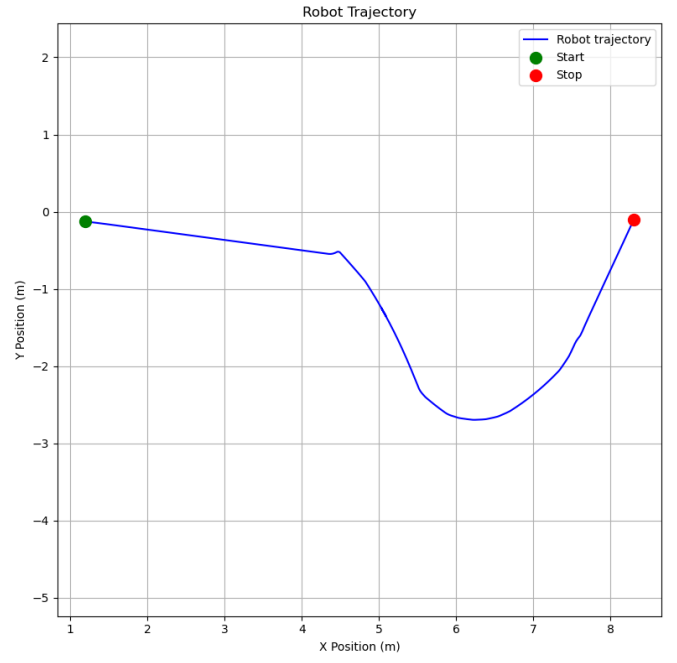


Fig. 5. Robot trajectory during a complete mission run, from starting position to the center of the circular room.

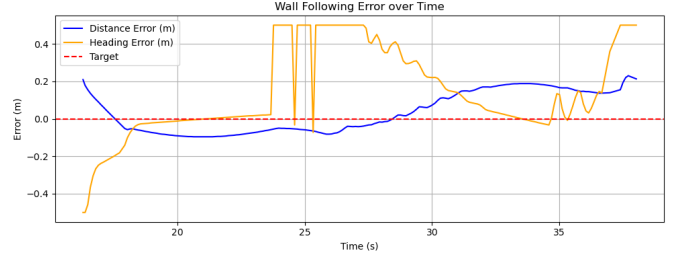


Fig. 6. Wall-following distance and heading error over time.

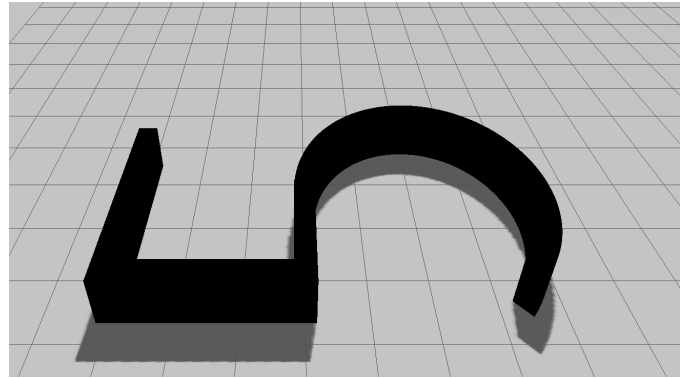


Fig. 8. Second 5-shaped map used for validation

REFERENCES

- [1] R. R. Murphy, *Introduction to AI Robotics*. Cambridge, MA: MIT Press, 2000.
- [2] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14-23, March 1986.

- [3] R. C. Arkin, *Behavior-Based Robotics*. Cambridge, MA: MIT Press, 1998.